



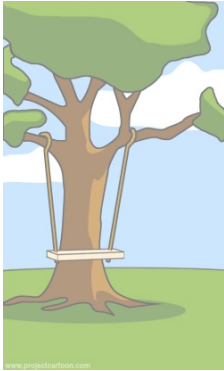
Software Engineering

3. System Modeling | Matthias Tichy, Sabrina Böhm, Timo Zuccarello | October 22, 2024

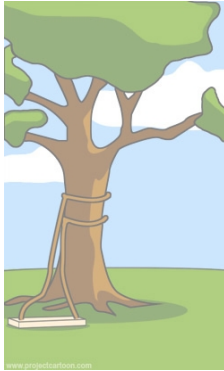
Why System Modeling?



how the customer explained it



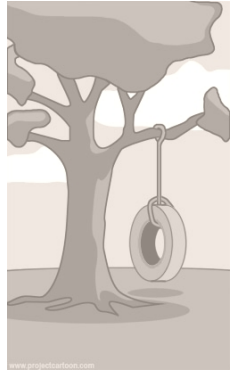
how the project leader understood it



how the programmer implemented it



how the business consultant described it



what the customer really needed

Lecture Overview – 3. System Modeling

- 1. Why to Model Systems?**
- 2. Understanding the Domain**
- 3. Modeling Behavior with Activity Diagrams**
- 4. Modeling Behavior with State Machines**

3. System Modeling

1. Why to Model Systems?

Motivation for Modeling

Recap: Software Engineering vs Programming

What is System Modeling?

What is a Model?

What Language to Use for Modeling?

The Unified Modeling Language (UML)

Different Kinds of UML Diagrams

14 Types of UML Diagrams

Lessons Learned

2. Understanding the Domain

3. Modeling Behavior with Activity Diagrams

Motivation for Modeling

UML User Guide:

“A successful software organization is one that consistently deploys **quality software** that meets the needs of its users. An organization that can develop such software in a **timely and predictable** fashion, with an **efficient and effective use of resources**, both human and material, is one that has a sustainable business.

[...]

Modeling is a central part of all the activities that lead up to the deployment of good software. We build models to **communicate** the desired structure and behavior of our system. We build models to **visualize and control** the system's architecture. We build models to better **understand** the system we are building, often exposing opportunities for **simplification and reuse**. And we build models to **manage risk**.”

UML User Guide:

“We build models of complex systems because we cannot comprehend such a system in its entirety.”

Recap: Software Engineering vs Programming



what is needed besides programming will be motivated and shown throughout this course



Bjarne Stroustrup (2000)

[C++]

“The most important single aspect of software development is to be clear about what you are trying to build.”

System Modeling

System Modeling

[Sommerville]

“**System modeling** is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system. [...] Models are used during the requirements engineering process to help derive the **detailed requirements** for a system, during the design process to **describe the system to engineers** implementing the system, and after implementation to **document the system**’s structure and operation.”

Stachowiak - Model Theory

H. Stachowiak. Allgemeine Modelltheorie, Springer, 1973.

A formal representation of entities and relationships in the real world (**abstraction**) with a certain correspondence (**homomorphism**) for a certain purpose (**pragmatics**).

Pragmatics

Create the model goal-driven

Abstraction

Remove details which do not serve the purpose

Homomorphism

Statements on model elements hold for real world entities

→ A model can replace the original under certain assumptions or for specific questions

What is a Model?

UML User Guide:

“A **model** is a simplification of reality.”

Sommerville:

“A **model** is an abstract view of a system that deliberately ignores some system details.”

Goals of Models

[UML User Guide]

- visualize a system as it is (wanted)
- specify the structure or behavior of a system
- template to guide construction of a system
- document the decisions we have made

Sommerville:

“It is important to understand that a system model is **not a complete representation** of system. It purposely leaves out detail to make it **easier to understand**. A model is an abstraction of the system being studied rather than an alternative representation of that system. A representation of a system should maintain all the information about the entity being represented. An abstraction **deliberately simplifies a system** design and picks out the most salient characteristics.”

What Language to Use for Modeling?

Towards a Common Language

- Natural language? hard to abstract from details, already used in requirements
- Programming language? unfamiliar to people without programming skills in that language, too early to decide for the programming language
- Textual language? harder to understand
- Graphical language? makes use of our visual abilities, requires common understanding
- Problem: engineers need to be aware of all languages being used
- Solution: use a graphical language independent of company and domain

HOW STANDARDS PROLIFERATE:

(SEE: A/C CHARGERS, CHARACTER ENCODINGS, INSTANT MESSAGING, ETC.)

SITUATION:
THERE ARE
14 COMPETING
STANDARDS.

14?! RIDICULOUS!
WE NEED TO DEVELOP
ONE UNIVERSAL STANDARD
THAT COVERS EVERYONE'S
USE CASES.



SOON:

SITUATION:
THERE ARE
15 COMPETING
STANDARDS.

The Unified Modeling Language (UML)

UML

[UML Reference Manual]

“The Unified Modeling Language (UML) is a general-purpose visual modeling language that is used to specify, visualize, construct, and document the artifacts of a software system.”

UML User Guide:

“Modeling yields an understanding of a system. No one model is ever sufficient. Rather, you often need multiple models that are connected to one another [...]”

Different Kinds of UML Diagrams

Structure Diagrams (Strukturdiagramme)

“**Structure diagrams** show the static structure of the objects in a system. That is, they depict those elements in a specification that are irrespective of time. The elements in a structure diagram represent the meaningful concepts of an application, and may include abstract, real-world and implementation concepts.”

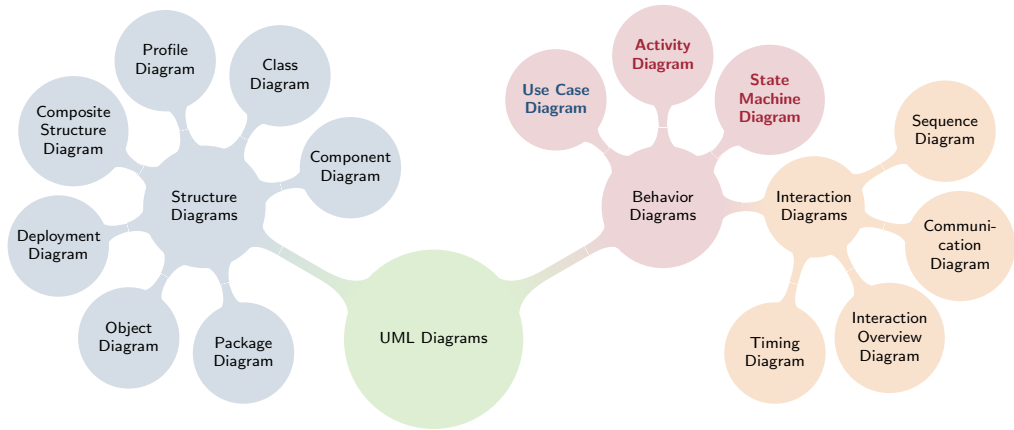
[UML 2.5.1]

Behavior Diagrams (Verhaltensdiagramme)

“**Behavior diagrams** show the dynamic behavior of the objects in a system, including their methods, collaborations, activities, and state histories. The dynamic behavior of a system can be described as a series of changes to the system over time.”

[UML 2.5.1]

14 Types of UML Diagrams [UML 2.5.1]



Six most important UML diagrams* discussed in this course

* John Erickson and Keng Siau. 2007. Theoretical and practical complexity of modeling methods. Commun. ACM 50, 8 (August 2007), 46–51.

Why to Model Systems?

Lessons Learned

- What is the motivation for system modeling?
- What are models and what is UML?
- Which (kinds of) UML diagrams exist?
- Further Reading: **UML User Guide**, Chapter 1 — great introduction to modeling

Practice

1. Practice abstraction by sketching a room that you know well for 1 minute (not more!).
2. Show the sketch to your group (2-3 students) and discuss what they abstract from (i.e., what details are not shown in the visualization). (5 min)

3. System Modeling

1. Why to Model Systems?

2. Understanding the Domain

Domain Models

3. Modeling Behavior with Activity Diagrams

4. Modeling Behavior with State Machines

Domain Models

Domain Modell (Domänenmodell)

A **domain model** covers the relevant elements and their relationships of the application domain to support mutual understanding

Rules for Domain Models

- static structure in the form of a class diagram
- often no operations
- if data types, only with types from the domain
- do **not** think in terms of technical design/implementation

Reminder: Class Diagram

Class Diagram

[UML User Guide]

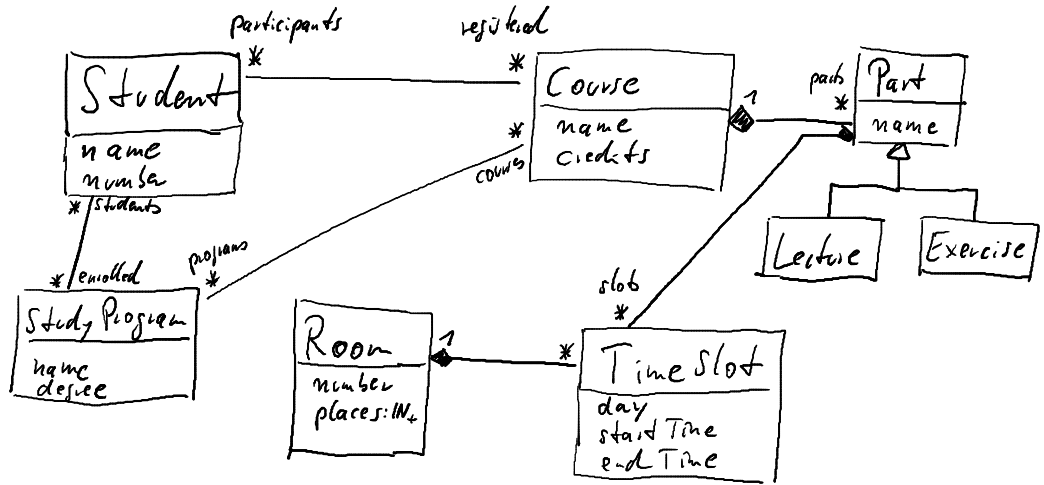
“A **class** is a description of a set of objects that share the same attributes, operations, relationships, and semantics. [...] An **association** is a structural relationship that specifies that objects of one thing are connected to objects of another. [...] When a class participates in an association, it has a specific role that it plays in that relationship; a **role** is just the face the class at the far end of the association presents to the class at the near end of the association. [...] When you state a **multiplicity** at the far end of an association, you are specifying that, for each object of the class at the near end, how many objects at the [far] end may exist.”

Example Multiplicities (default=*)

[UML Reference Manual]

0..* (=*) or 1..* or 0..1 or 1..1 (=1) ... 2..5 ...

Example Domain Model



Live exercise: Domain Models

Practice

- Join forces with your neighbours (2-3 persons)
- Identify classes, attributes and associations
- Sketch a domain model visualized as a class diagram

Text

“As researchers in academia, one common way to publish your results is to write papers (scientific articles). As researchers, similarly to employees in industry, typically cooperate with other researchers during their work, papers usually have multiple co-authors who jointly worked on the paper presented within them. A paper can be published by submitting it to a conference. In order to ensure a certain level of quality, papers are peer-reviewed by other researchers. Every conference has a program committee, which consists of a number of researchers knowledgeable in the area of research the conference covers. Members of the program committee are assigned to review all submitted papers, with at least three reviewers for every paper. Every review has a grade between -2 (reject) and 2 (accept). A paper can be in submission, accepted or rejected.”

3. System Modeling

1. Why to Model Systems?

2. Understanding the Domain

3. Modeling Behavior with Activity Diagrams

Activity Diagrams

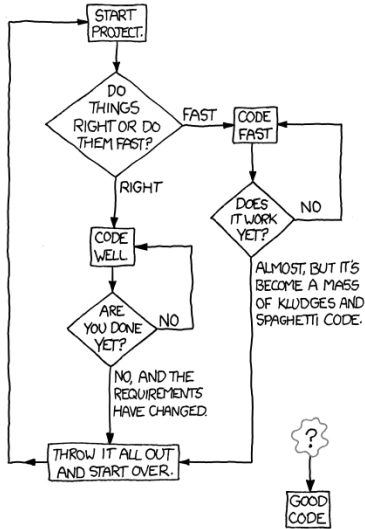
Branching and Merging in Activity Diagrams

Forking and Joining in Activity Diagrams

Swimlanes in Activity Diagrams

4. Modeling Behavior with State Machines

HOW TO WRITE GOOD CODE:



Activity Diagrams



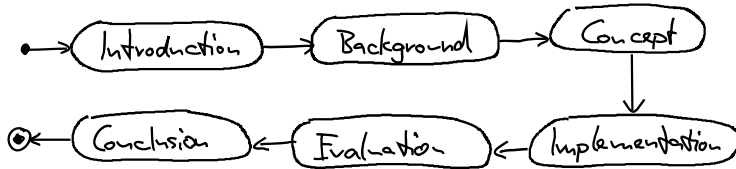
Activity Diagram (Aktivitätsdiagramm)

An **activity diagram** is a diagram visualizing activities and their order of execution. An activity diagram contains **activities** (rounded box) that are connected by means of **flows** (solid arrows). The execution begins at the **initialization** (filled circle) and ends with the **completion** node (bull's eye).
(Aktivität, Fluss, Startzustand, Endzustand)

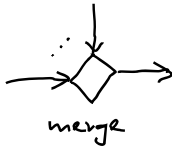
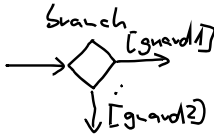
Rules for Activity Diagrams

- exactly one initialization/completion node
- at least one activity
- every activity has one incoming and one outgoing flow
- every activity is reachable from initialization
- completion is reachable from every activity

Example of Sequential Activities



Branching and Merging in Activity Diagrams



Branching and Merging

[UML User Guide]

Motivation: model control flow that depends on certain conditions (i.e., actions that may happen)

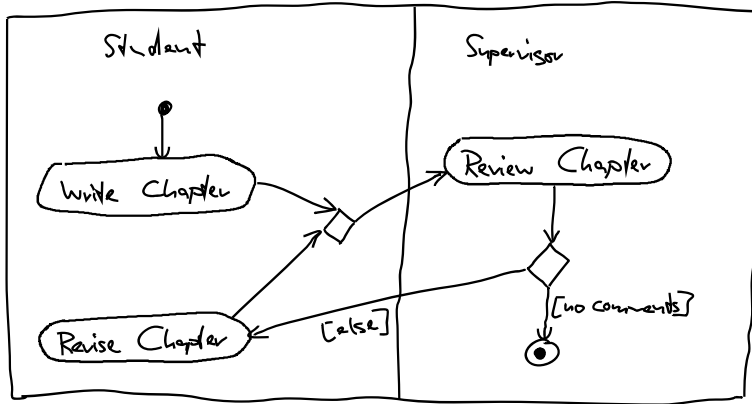
Branching: A **branch** has exactly one incoming and two or more outgoing flows. Each outgoing flow has a Boolean expression called **guard**, which is evaluated on entering the branch. (Verzweigung)

Merging: A **merge** has two or more incoming and exactly one outgoing flow. (Zusammenführung)

Further Rules for Activity Diagrams

- guards on outgoing flows should not overlap (flow of control is unambiguous)
- guards should cover all possibilities (flow of control does not freeze)
- keyword **else** possible for one guard (sonst)

Example of Conditional Activities



Forking and Joining in Activity Diagrams



Forking and Joining

[UML User Guide]

Motivation: model concurrent control flows (i.e., activities that run in parallel)

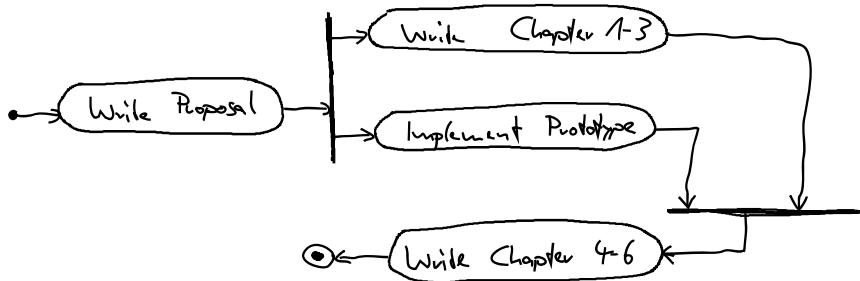
Forking: A **fork** (thick horizontal or vertical line) has exactly one incoming and two or more outgoing flows. (Gabelung)

Joining: A **join** (thick horizontal or vertical line) has two or more incoming and exactly one outgoing flow. (Vereinigung)

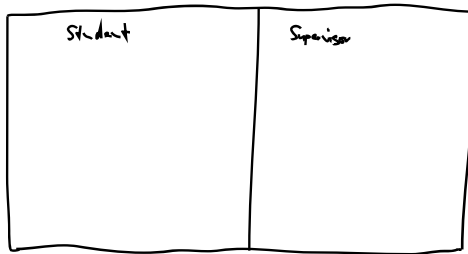
Further Rules for Activity Diagrams

- branched paths must be merged eventually (letztendlich)
- forked paths must be joined eventually
- only outgoing edges of branch nodes have guards

Example of Concurrent Activities



Swimlanes in Activity Diagrams



Swimlanes

[UML User Guide]

Motivation: group activities according to responsibilities

Swimlane: An activity diagram may have no or at least two swimlanes. A **swimlane** (rectangle) represents a high-level responsibility activities within an activity diagram. (*Verantwortlichkeitsbereiche*)

Further Rules for Activity Diagrams

- each swimlane has a name unique within its diagram
- every activity belongs to exactly one swimlane
- only flows may cross swimlanes

Live exercise: Activity Diagram

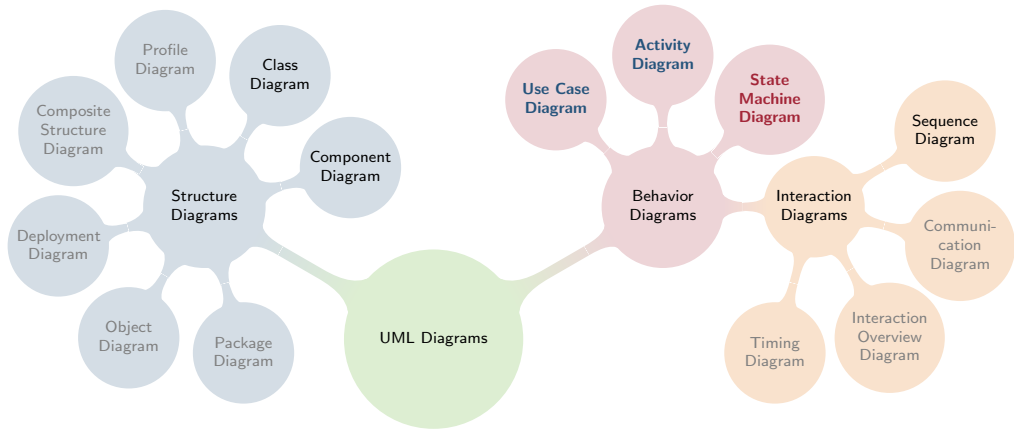
Practice

- Join forces with your neighbours (2-3 persons)
- Sketch an activity diagram that shows the different activities, transitions and swim lane

Extension of earlier text

“One of the authors of a scientific paper submits the paper to a conference. It is also possible to submit an abstract first and then the full paper. As long as the possibility to submit papers has not been disabled, it is possible to submit new versions of the paper. After submission, the papers are distributed to the reviewers (members of the Program Committee). This is done either by the PC Chairs manually assigning the papers to reviewers (the number of reviewers per paper is set beforehand) or by each reviewer deciding for each paper whether they can review the paper in principle and then the system carries out an automatic assignment based on this information. In the case of automatic assignment, it is still possible to change the assignment manually. Once the reviews have been created, they are uploaded to the system. As soon as all reviews for a paper have been received (and the PC Chairs have activated this functionality), the paper and the associated reviews are discussed by the reviewers in a kind of forum. Finally, the PC Chairs decide for each paper whether it should be accepted or rejected. Depending on the decision, a rejection or acceptance e-mail is sent by the PC Chairs.”

Recap: 14 Types of UML Diagrams [UML 2.5.1]



Lessons Learned

- What can be modeled with activity diagrams?
- What are branching and merging (used for)?
- What are forking and joining (used for)?
- What can be modeled with swimlanes?
- Further Reading: [UML User Guide](#), Chapter 20

3. System Modeling

1. Why to Model Systems?

2. Understanding the Domain

3. Modeling Behavior with Activity Diagrams

4. Modeling Behavior with State Machines

Activity and State Machine Diagrams

State Machine Diagrams

Hierarchical State Machine Diagrams

Lessons Learned

Activity and State Machine Diagrams

UML User Guide:

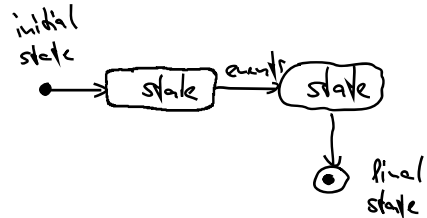
We can visualize the dynamics of execution in two ways: by emphasizing the flow of control from activity to activity (**activity diagrams**) or by emphasizing the potential states and transitions among those states (**state machine diagrams**).

State Machine Diagrams

State Machine Diagram (Zustandsdiagramm)

A **state machine diagram** specifies the sequences of states the (a part of) the system goes through during its lifetime in response to events, together with its responses to those events. Every **state** (rectangle with rounded corners) is characterized by a condition or situation. An **event** is an occurrence of a stimulus that can trigger a state transition. A **transition** (solid arrow) is a relationship between two states. (Zustand, Ereignis, Zustandsübergang)

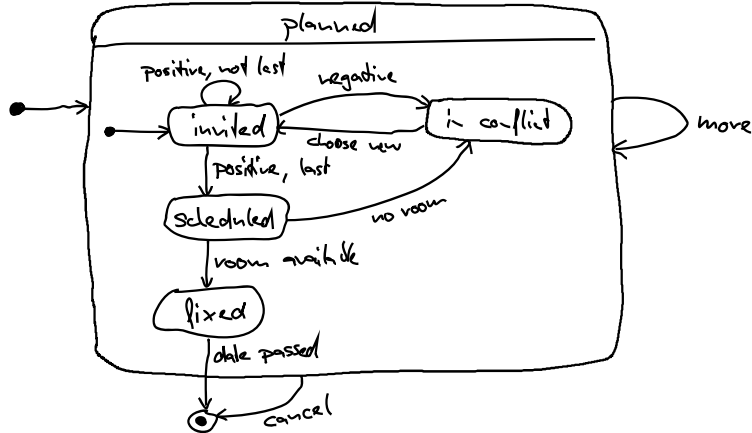
[adapted from UML User Guide]



Rules for State Machine Diagrams

there is a single **initial state** (filled circle) and a single **final state** (bull's eye) (Start- und Zielzustand) — see exception below

Example of a State Machine Diagram



Hierarchical State Machine Diagrams

Simple and Composite States

[UML User Guide]

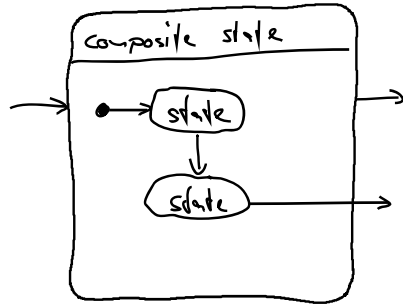
Motivation: avoid duplicated transitions, improve overview in complex state machine diagrams

Simple State: “A **simple state** is a state that has no substructure.” (einfacher Zustand)

Composite State: “A state that has substates (i.e., nested states) is called a **composite state**.” (komplexer Zustand)

Rules for State Machine Diagrams

- every composite state has its own single **initial state** (Startzustand)
- substates may be nested to any level



Live exercise: State Machine Diagram

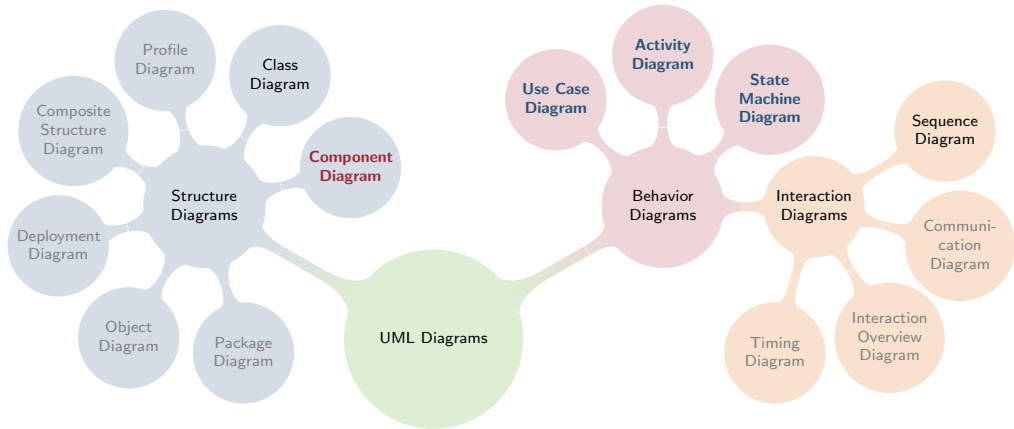
Practice

- Join forces with your neighbours (2-3 persons)
- Sketch a state machine diagram that shows the different states of a paper during all activities

Extension of earlier text

“One of the authors of a scientific paper submits the paper to a conference. It is also possible to submit an abstract first and then the full paper. As long as the possibility to submit papers has not been disabled, it is possible to submit new versions of the paper. After submission, the papers are distributed to the reviewers (members of the Program Committee). This is done either by the PC Chairs manually assigning the papers to reviewers (the number of reviewers per paper is set beforehand) or by each reviewer deciding for each paper whether they can review the paper in principle and then the system carries out an automatic assignment based on this information. In the case of automatic assignment, it is still possible to change the assignment manually. Once the reviews have been created, they are uploaded to the system. As soon as all reviews for a paper have been received (and the PC Chairs have activated this functionality), the paper and the associated reviews are discussed by the reviewers in a kind of forum. Finally, the PC Chairs decide for each paper whether it should be accepted or rejected. Depending on the decision, a rejection or acceptance e-mail is sent by the PC Chairs.”

Recap: 14 Types of UML Diagrams [UML 2.5.1]



Modeling Behavior with State Machines

Lessons Learned

- What can be modeled with state machine diagrams?
- What is the advantage of hierarchical state machines?
- Further Reading: [UML User Guide](#), Chapter 22

Practice

1. Draw a simple state machine diagram for an Instant Messenger with a single mistake and show it to your neighbors.
2. Find the mistake in the diagrams of your neighbors and discuss how to correct it.